Superstore Sales Analysis

Objective

Analyze retail sales data to identify key drivers of profitability and losses.
The analysis focuses on understanding how discount levels impact profit margins and aims to provide data-driven recommendations for improving pricing and discount strategies.

In [211…

```
import pandas as pd
import numpy as np

df = pd.read_csv("SuperStoreOrders.csv")
```

In [60]:

```
df.shape
```

Out[60]:

```
(51290, 21)
```

In [61]:

```
df.columns
```

Out[61]:

```
Index(['order_id', 'order_date', 'ship_date', 'ship_mode', 'customer_name',
       'segment', 'state', 'country', 'market', 'region', 'product_id',
       'category', 'sub_category', 'product_name', 'sales', 'quantity',
       'discount', 'profit', 'shipping_cost', 'order_priority', 'year'],
      dtype='object')
```

In [62]:

```
df.head()
```

Out[62]:

	order_id	order_date	ship_date	ship_mode	customer_name	segment	state	country	market	region	...	category	su
0	AG-2011-2040	1/1/2011	6/1/2011	Standard Class	Toby Braunhardt	Consumer	Constantine	Algeria	Africa	Africa	...	Office Supplies	
1	IN-2011-47883	1/1/2011	8/1/2011	Standard Class	Joseph Holt	Consumer	New South Wales	Australia	APAC	Oceania	...	Office Supplies	
2	HU-2011-1220	1/1/2011	5/1/2011	Second Class	Annie Thurman	Consumer	Budapest	Hungary	EMEA	EMEA	...	Office Supplies	
3	IT-2011-3647632	1/1/2011	5/1/2011	Second Class	Eugene Moren	Home Office	Stockholm	Sweden	EU	North	...	Office Supplies	
4	IN-2011-47883	1/1/2011	8/1/2011	Standard Class	Joseph Holt	Consumer	New South Wales	Australia	APAC	Oceania	...	Furniture	

5 rows × 21 columns

In [63]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51290 entries, 0 to 51289
Data columns (total 21 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   order_id        51290 non-null  object
 1   order_date      51290 non-null  object
 2   ship_date       51290 non-null  object
 3   ship_mode       51290 non-null  object
 4   customer_name   51290 non-null  object
 5   segment         51290 non-null  object
 6   state           51290 non-null  object
 7   country         51290 non-null  object
 8   market          51290 non-null  object
 9   region          51290 non-null  object
 10  product_id      51290 non-null  object
 11  category        51290 non-null  object
 12  sub_category    51290 non-null  object
 13  product_name    51290 non-null  object
 14  sales           51290 non-null  object
 15  quantity        51290 non-null  int64
 16  discount        51290 non-null  float64
 17  profit          51290 non-null  float64
 18  shipping_cost   51290 non-null  float64
 19  order_priority  51290 non-null  object
 20  year            51290 non-null  int64
dtypes: float64(3), int64(2), object(16)
memory usage: 8.2+ MB
```

In [88]:
```
df.isnull().sum()
```

Out[88]:
```
order_id          0
order_date        0
ship_date         0
ship_mode         0
customer_name     0
segment           0
state             0
country           0
market            0
region            0
product_id        0
category          0
sub_category      0
product_name      0
sales             0
quantity          0
discount          0
profit            0
shipping_cost     0
order_priority    0
year              0
order_year        0
order_month       0
shipping_days     0
profit_margin     1
dtype: int64
```

Data Overview

The dataset contains 51,290 rows and 21 columns. There are no missing values across all variables. Numeric fields such as sales, profit, discount, and shipping cost are correctly formatted. Date fields will be converted to datetime for time-based analysis.

In [64]:
```
df.dtypes
```

Out[64]:
```
order_id           object
order_date         object
ship_date          object
ship_mode          object
customer_name      object
segment            object
state              object
country            object
market             object
region             object
product_id         object
category           object
sub_category       object
product_name       object
sales              object
quantity            int64
discount          float64
profit            float64
shipping_cost     float64
order_priority     object
year                int64
dtype: object
```

The sales column will be cleaned by removing formatting characters (thousands separators) and converted to numeric format.

In [65]:
```
df['sales'].head(10)
```

Out[65]:
```
0    408
1    120
2     66
3     45
4    114
5     55
6    314
7    276
8    912
9    667
Name: sales, dtype: object
```

In [66]:
```
df['sales'].map(type).value_counts()
```

Out[66]:
```
sales
<class 'str'>    51290
Name: count, dtype: int64
```

In [67]:
```
pd.to_numeric(df['sales'], errors='coerce').isna().sum()
```

Out[67]:
```
np.int64(2630)
```

In [68]:
```
df[pd.to_numeric(df['sales'], errors='coerce').isna()]['sales'].head(20)
```

Out[68]:
```
27     1,648
43     3,029
58     2,574
83     2,479
84     2,174
100    2,875
125    1,384
126    1,323
163    1,200
173    1,326
213    1,412
257    2,797
258    1,244
259    1,068
302    1,102
309    1,303
342    1,113
413    2,571
414    1,917
450    1,052
Name: sales, dtype: object
```

In [69]:
```
df['sales'] = df['sales'].str.replace(',', '', regex=False)
```

In [70]:
```
df['sales'] = pd.to_numeric(df['sales'], errors='coerce')
```

In [71]:
```
df['sales'].isna().sum()
```

Out[71]:
```
np.int64(0)
```

In [90]:
```
df['sales'].dtype
```

Out[90]:
```
dtype('int64')
```

In [77]:
```
df['order_date'].head(10)
```

Out[77]:
```
0    1/1/2011
1    1/1/2011
2    1/1/2011
3    1/1/2011
4    1/1/2011
5    1/1/2011
6    2/1/2011
7    3/1/2011
8    3/1/2011
9    3/1/2011
Name: order_date, dtype: object
```

In [78]:
```
df['order_date'].sample(10)
```

Out[78]:
```
27696    29-08-2013
44666    19-09-2014
12011    28-05-2012
43225    28-08-2014
750      24-02-2011
9681      10/2/2012
13010    23-06-2012
11015    18-04-2012
45250    29-09-2014
6786      4/11/2011
Name: order_date, dtype: object
```

In [93]:
```
df['order_date'] = pd.to_datetime(df['order_date'], format="mixed", dayfirst=True)
df['ship_date'] = pd.to_datetime(df['ship_date'], format="mixed", dayfirst=True)
```

In [94]:
```
df[['order_date', 'ship_date']].isna().sum()
```

Out[94]:
```
order_date    0
ship_date     0
dtype: int64
```

The dataset contained mixed date formats. Dates were standardized using pandas mixed-format parsing with day-first logic to ensure correct temporal analysis without data loss.

In [92]:
```
df[['order_date', 'ship_date']].describe()
```

Out[92]:

	order_date	ship_date
count	51290	51290
mean	2013-05-11 21:26:49.155780864	2013-05-15 20:42:42.745174528
min	2011-01-01 00:00:00	2011-01-03 00:00:00
25%	2012-06-19 00:00:00	2012-06-23 00:00:00
50%	2013-07-08 00:00:00	2013-07-12 00:00:00
75%	2014-05-22 00:00:00	2014-05-26 00:00:00
max	2014-12-31 00:00:00	2015-01-07 00:00:00

In [82]:
```
df['order_year'] = df['order_date'].dt.year
df['order_month'] = df['order_date'].dt.to_period('M')

df[['order_date', 'order_year', 'order_month']].head()
```

Out[82]:

	order_date	order_year	order_month
0	2011-01-01	2011	2011-01
1	2011-01-01	2011	2011-01
2	2011-01-01	2011	2011-01
3	2011-01-01	2011	2011-01
4	2011-01-01	2011	2011-01

In [95]:
```
print("order_date type:", df['order_date'].dtype)
print("Earliest date:", df['order_date'].min())
print("Latest date:", df['order_date'].max())
```

```
order_date type: datetime64[ns]
Earliest date: 2011-01-01 00:00:00
Latest date: 2014-12-31 00:00:00
```

In [96]:
```
df['shipping_days'] = (df['ship_date'] - df['order_date']).dt.days
```

In [97]:
```
df['shipping_days'].describe()
```

Out[97]:
```
count    51290.000000
mean         3.969370
std          1.729437
min          0.000000
25%          3.000000
50%          4.000000
75%          5.000000
max          7.000000
Name: shipping_days, dtype: float64
```

In [98]:
```
df[['sales', 'profit', 'discount', 'quantity']].describe()
```

Out[98]:

	sales	profit	discount	quantity
count	51290.000000	51290.000000	51290.000000	51290.000000
mean	246.498440	28.641740	0.142908	3.476545
std	487.567175	174.424113	0.212280	2.278766
min	0.000000	-6599.978000	0.000000	1.000000
25%	31.000000	0.000000	0.000000	2.000000
50%	85.000000	9.240000	0.000000	3.000000
75%	251.000000	36.810000	0.200000	5.000000
max	22638.000000	8399.976000	0.850000	14.000000

Key Performance Metrics

In [190…
```
loss_orders = (df['profit'] < 0).sum()
loss_rate = (df['profit'] < 0).mean()

loss_orders, loss_rate
```

Out[190…
```
(np.int64(12543), np.float64(0.24455059465782802))
```

In [216…
```
total_sales = df_margin['sales'].sum()
total_profit = df_margin['profit'].sum()
profit_margin = total_profit / total_sales
loss_rate = (df_margin['profit'] < 0).mean()
avg_discount = df_margin['discount'].mean()
total_orders = df_margin['order_id'].nunique()

print(f"Total Sales: {total_sales:,.0f}")
print(f"Total Profit: {total_profit:,.0f}")
print(f"Profit Margin: {profit_margin:.2%}")
print(f"Loss Making Orders: {loss_rate:.2%}")
print(f"Average Discount: {avg_discount:.2%}")
print(f"Total Orders: {total_orders}")
```

```
Total Sales: 12,642,905
Total Profit: 1,469,036
Profit Margin: 11.62%
Loss Making Orders: 24.45%
Average Discount: 14.29%
Total Orders: 25035
```

Loss-Making Orders

Out of all orders, **12,543 orders (~24%) generate a loss**.

This means roughly **1 out of every 4 orders is unprofitable**, indicating a significant profitability issue.

Profit results suggest a potential profitability issue. A root cause analysis will be performed to identify the factors driving losses.

In [119…
```
df['profit_margin'] = df['profit'] / df['sales']
```

In [120…
```
(df['sales'] == 0).sum()
```

Out[120… np.int64(1)

The record with zero sales will be removed from margin analysis because margin cannot be calculated when sales is zero.

In [128…
```
df['profit_margin'] = df['profit_margin'].replace([np.inf, -np.inf], np.nan)
```

In [132…
```
df['profit_margin'].isna().sum()
```

Out[132… np.int64(1)

In [147…
```
df_margin = df[df['sales'] > 0].copy()
```

In [135…
```
df_margin.shape
```

Out[135… (51289, 25)

In [136…
```
df.shape
```

Out[136… (51290, 25)

In [124…
```
df['profit_margin'].describe()
```

Out[124…
```
count    51289.000000
mean         0.047208
std          0.466851
min         -4.727752
25%          0.000000
50%          0.169257
75%          0.333136
max          0.582800
Name: profit_margin, dtype: float64
```

In [154…
```
df_margin['discount_band'] = pd.cut(
    df_margin['discount'],
    bins=[-0.001, 0, 0.1, 0.2, 0.3, 0.5, 1.0],
    labels=['0', '0-10%', '10-20%', '20-30%', '30-50%', '50%+']
)
```

In [155…
```
df_margin.groupby('discount_band', observed=True)['profit_margin'].agg(['count','mean','median'])
```

Out[155…

discount_band	count	mean	median
0	29009	0.264827	0.270000
0-10%	4679	0.171421	0.168285
10-20%	6274	0.137621	0.124904
20-30%	967	-0.038533	-0.042850
30-50%	6189	-0.337888	-0.331475
50%+	4171	-1.150365	-1.065715

In [157…
```
df_margin.groupby('discount_band', observed=True).agg({'profit_margin': ['count','mean','median']})
```

Out[157…

	profit_margin		
discount_band	count	mean	median
0	29009	0.264827	0.270000
0-10%	4679	0.171421	0.168285
10-20%	6274	0.137621	0.124904
20-30%	967	-0.038533	-0.042850
30-50%	6189	-0.337888	-0.331475
50%+	4171	-1.150365	-1.065715

Profitability declines as discount levels increase.

Up to 20% discount, profit margins remain positive.

However, above 20% discount, the average profit margin turns negative, suggesting that higher discount levels are likely driving losses.

In [160…

```
(df_margin.groupby('discount_band', observed=True)
 .agg(
      orders=('order_id','count'),
      sales=('sales','sum'),
      profit=('profit','sum'),
      avg_margin=('profit_margin','mean')
 )
 .sort_index()
)
```

Out[160…

discount_band	orders	sales	profit	avg_margin
0	29009	6992734	1.770695e+06	0.264827
0-10%	4679	1962633	3.397668e+05	0.171421
10-20%	6274	1757296	1.732548e+05	0.137621
20-30%	967	382540	-2.115561e+04	-0.038533
30-50%	6189	1176078	-3.809448e+05	-0.337888
50%+	4171	371624	-4.125805e+05	-1.150365

In [163…

```
pd.options.display.float_format = '{:,.2f}'.format
```

In [164…

```
(df_margin.groupby('discount_band', observed=True)
 .agg(
      orders=('order_id','count'),
      sales=('sales','sum'),
      profit=('profit','sum'),
      avg_margin=('profit_margin','mean')
 )
 .sort_index()
)
```

Out[164…

discount_band	orders	sales	profit	avg_margin
0	29009	6992734	1,770,695.27	0.26
0-10%	4679	1962633	339,766.79	0.17
10-20%	6274	1757296	173,254.84	0.14
20-30%	967	382540	-21,155.61	-0.04
30-50%	6189	1176078	-380,944.82	-0.34
50%+	4171	371624	-412,580.55	-1.15

Discounts above 30% generate substantial total losses (approximately 793K in combined losses), indicating an unsustainable discount strategy.

In [179…

```
loss_by_subcat = (high_disc
    .groupby('sub_category')
    .agg(
        orders=('order_id','count'),
        sales=('sales','sum'),
        profit=('profit','sum'),
        avg_discount=('discount','mean')
    )
    .sort_values('profit', ascending=True)
)

loss_by_subcat.head(10)
```

Out[179…

sub_category	orders	sales	profit	avg_discount
Tables	467	334025	-132,541.15	0.46
Bookcases	572	203131	-90,436.17	0.48
Phones	631	177914	-87,190.86	0.50
Chairs	683	187573	-72,214.88	0.46
Machines	346	136496	-71,799.09	0.56
Storage	987	125154	-62,799.43	0.51
Copiers	421	157547	-59,345.20	0.48
Appliances	306	95830	-58,088.63	0.58
Binders	1446	57985	-51,905.67	0.63
Accessories	577	81887	-37,400.06	0.49

In [194…

```
high_disc = df_margin[df_margin['discount'] >= 0.3].copy()

loss_by_reg = (high_disc
    .groupby(['region','state'])
    .agg(
        orders=('order_id','count'),
        sales=('sales','sum'),
        profit=('profit','sum'),
        avg_discount=('discount','mean'),
        avg_margin=('profit_margin','mean')
    )
    .sort_values('profit', ascending=True)
)

loss_by_reg.head(10)
```

Out[194…

region	state	orders	sales	profit	avg_discount	avg_margin
Central	Texas	414	91584	-33,954.17	0.60	-1.06
EMEA	Istanbul	425	31041	-29,033.70	0.60	-0.92
Africa	Lagos	333	17188	-25,922.51	0.70	-1.45
East	Ohio	179	49476	-20,227.22	0.53	-0.48
Central Asia	Punjab	143	31418	-18,867.58	0.51	-0.52
Central	Illinois	228	37540	-18,436.99	0.61	-1.04
East	Pennsylvania	233	73193	-16,140.10	0.52	-0.45
EMEA	Izmir	123	15165	-15,729.80	0.60	-0.88
Central	Francisco Morazán	362	48061	-15,007.42	0.41	-0.30
Southeast Asia	National Capital	329	56495	-14,979.80	0.43	-0.34

Discounts above 30% are consistently generating losses.

The largest losses come from high-volume categories and aggressive discount regions such as EMEA and Africa.

In [181…

```
low_disc  = df_margin[df_margin['discount'] < 0.3].copy()
high_disc = df_margin[df_margin['discount'] >= 0.3].copy()
```

In [184…

```
low_by_subcat = (low_disc
    .groupby('sub_category')
    .agg(
        orders=('order_id','count'),
        sales=('sales','sum'),
        profit=('profit','sum'),
        avg_discount=('discount','mean'),
        avg_margin=('profit_margin','mean')
    )
    .sort_values('profit')
)

low_by_subcat.head(10)   #30% altı indirimde en çok zarar eden 10 sub_category
```

Out[184…

sub_category	orders	sales	profit	avg_discount	avg_margin
Fasteners	1832	70581	17,525.99	0.03	0.25
Labels	2085	64966	18,580.72	0.03	0.28
Supplies	1888	213526	37,286.06	0.02	0.22
Envelopes	1877	146233	40,027.81	0.02	0.27
Tables	394	423009	68,457.76	0.09	0.14
Paper	3027	224245	69,032.65	0.04	0.33
Furnishings	2449	335802	72,090.23	0.05	0.24
Art	4055	338356	77,795.97	0.03	0.23
Binders	4706	403967	124,355.51	0.04	0.27
Machines	1140	642575	130,666.96	0.05	0.19

In [183…

```
high_by_subcat = (high_disc
    .groupby('sub_category')
    .agg(
        orders=('order_id','count'),
        sales=('sales','sum'),
        profit=('profit','sum'),
        avg_discount=('discount','mean'),
        avg_margin=('profit_margin','mean')
    )
    .sort_values('profit')
)

high_by_subcat.head(10)
```

Out[183…

sub_category	orders	sales	profit	avg_discount	avg_margin
Tables	467	334025	-132,541.15	0.46	-0.57
Bookcases	572	203131	-90,436.17	0.48	-0.57
Phones	631	177914	-87,190.86	0.50	-0.59
Chairs	683	187573	-72,214.88	0.46	-0.51
Machines	346	136496	-71,799.09	0.56	-0.83
Storage	987	125154	-62,799.43	0.51	-0.66
Copiers	421	157547	-59,345.20	0.48	-0.56
Appliances	306	95830	-58,088.63	0.58	-1.06
Binders	1446	57985	-51,905.67	0.63	-0.90
Accessories	577	81887	-37,400.06	0.49	-0.53

Discount Policy Impact

Products such as Tables, Machines and Binders are profitable under normal discount levels. However, when discounts exceed 30%, the same products turn unprofitable.

This suggests that the main issue is aggressive discount strategy rather than product economics.

In [185…

```
high_disc['sales'].sum() / df_margin['sales'].sum()
```

Out[185…

```
np.float64(0.13735885858511157)
```

High-discount orders (30%+) represent only 14% of total sales, yet they generate significant losses.

This indicates that a relatively small portion of sales is driving a large share of the profitability problem.

In [186…

```
df_margin[['discount','profit_margin']].corr()
```

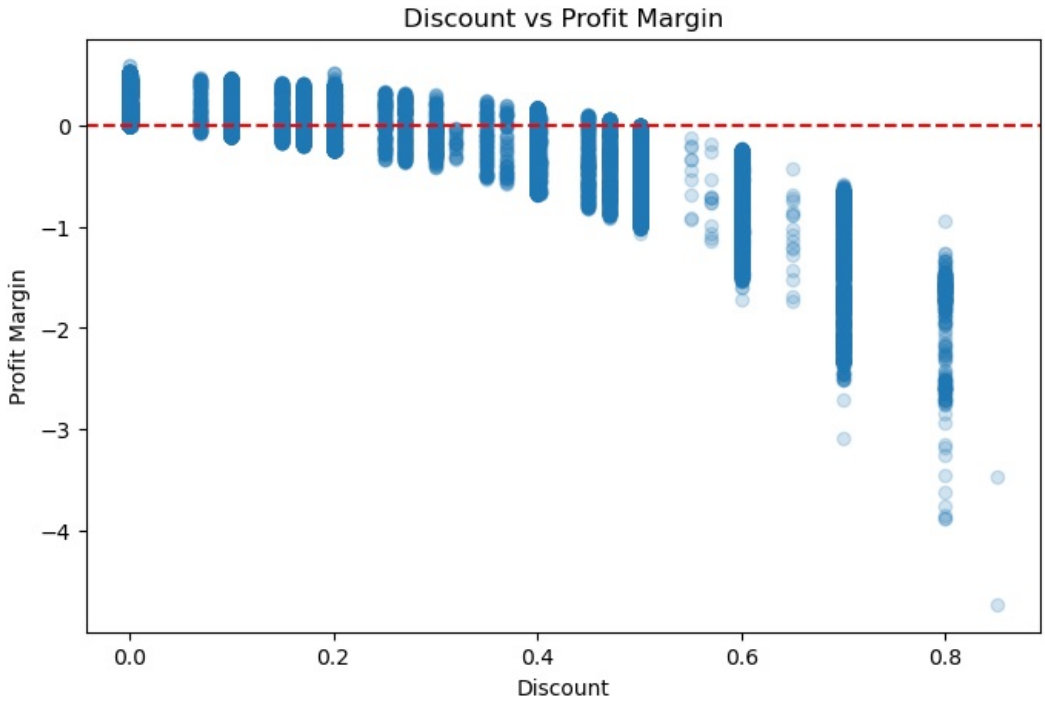
Out[186…

	discount	profit_margin
discount	1.00	-0.85
profit_margin	-0.85	1.00

In [187…

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(8,5))
plt.scatter(df_margin['discount'], df_margin['profit_margin'], alpha=0.2)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Discount')
plt.ylabel('Profit Margin')
plt.title('Discount vs Profit Margin')
plt.show()
```

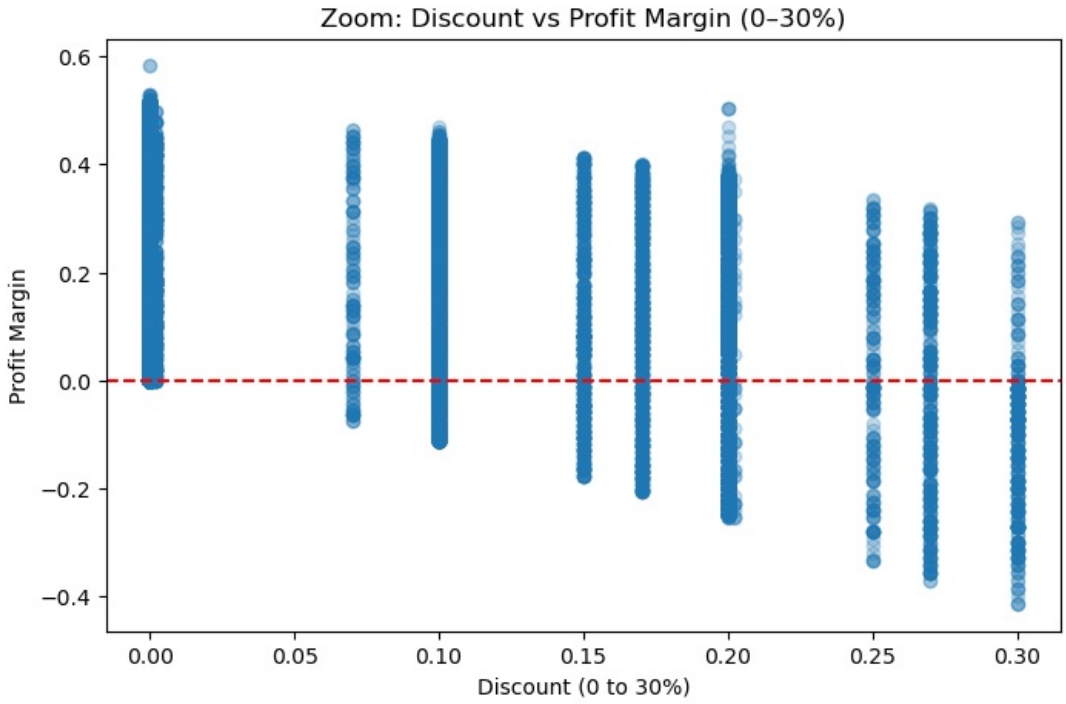


In [188…

```
import matplotlib.pyplot as plt

zoom = df_margin[df_margin['discount'] <= 0.3]

plt.figure(figsize=(8,5))
plt.scatter(zoom['discount'], zoom['profit_margin'], alpha=0.25)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Discount (0 to 30%)')
plt.ylabel('Profit Margin')
plt.title('Zoom: Discount vs Profit Margin (0–30%)')
plt.show()
```



In [189…

```
tmp = df_margin.copy()
tmp['disc_bin'] = pd.cut(tmp['discount'], bins=[0,0.05,0.10,0.15,0.20,0.25,0.30,0.40,0.50,0.60,0.70,0.80,1.0])

bin_summary = (tmp.groupby('disc_bin')
               .agg(orders=('order_id','count'),
                    median_margin=('profit_margin','median'),
```

```
                    mean_margin=('profit_margin','mean'))
                )

bin_summary
```

```
C:\Users\ceren\AppData\Local\Temp\ipykernel_27140\1671552476.py:4: FutureWarning: The default of observed=False
is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current b
ehavior or observed=True to adopt the future default and silence this warning.
  bin_summary = (tmp.groupby('disc_bin')
```

Out[189…

disc_bin	orders	median_margin	mean_margin
(0.0, 0.05]	461	0.20	0.23
(0.05, 0.1]	4218	0.17	0.17
(0.1, 0.15]	541	0.07	0.09
(0.15, 0.2]	5733	0.12	0.14
(0.2, 0.25]	239	-0.01	0.02
(0.25, 0.3]	728	-0.07	-0.06
(0.3, 0.4]	3400	-0.24	-0.25
(0.4, 0.5]	2789	-0.44	-0.45
(0.5, 0.6]	2028	-0.85	-0.87
(0.6, 0.7]	1826	-1.26	-1.34
(0.7, 0.8]	315	-1.66	-1.87
(0.8, 1.0]	2	-4.10	-4.10

Discount Threshold Analysis

Profit margins remain positive up to ~20% discount levels.
Beyond 20–25%, the median profit margin turns negative and losses increase rapidly.

This suggests that discount levels above ~20% significantly increase the risk of unprofitable sales.

In [195…

```
df_margin[['shipping_cost','profit_margin','discount']].corr()
```

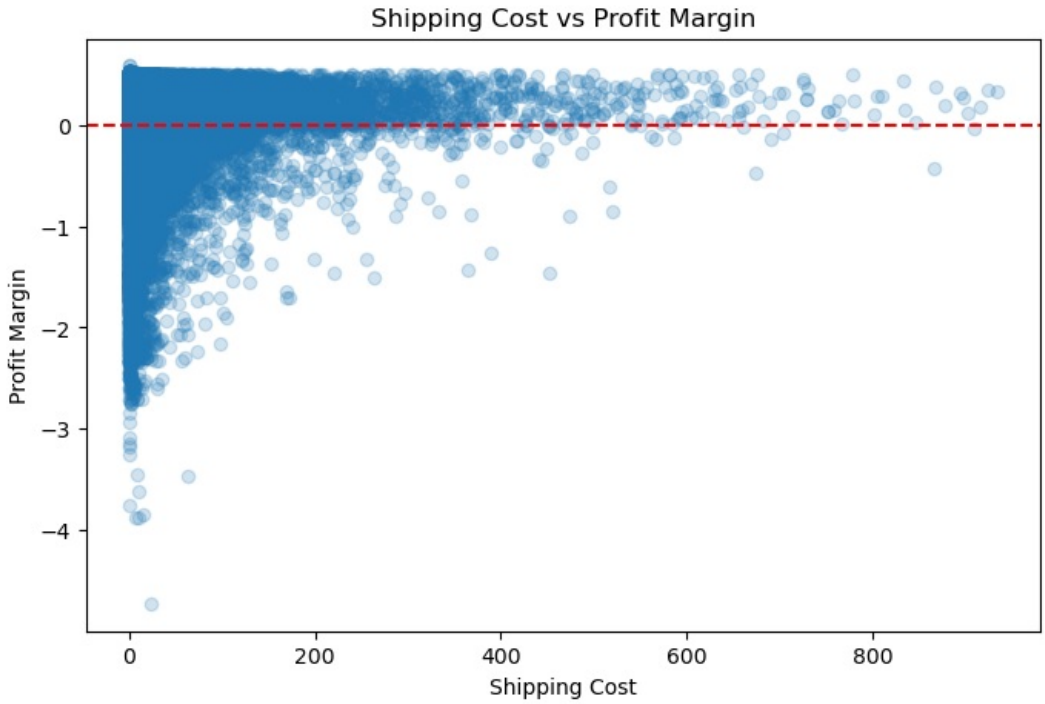
Out[195…

	shipping_cost	profit_margin	discount
shipping_cost	1.00	0.07	-0.08
profit_margin	0.07	1.00	-0.85
discount	-0.08	-0.85	1.00

In [196…

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))
plt.scatter(df_margin['shipping_cost'], df_margin['profit_margin'], alpha=0.2)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Shipping Cost')
plt.ylabel('Profit Margin')
plt.title('Shipping Cost vs Profit Margin')
plt.show()
```



In [202…

```
low_disc = df_margin[df_margin['discount'] <= 0.2].copy()

tmp = low_disc.copy()

tmp['ship_ratio_bin'] = pd.cut(
    tmp['shipping_cost_ratio'],
    bins=[0, 0.05, 0.10, 0.15, 0.20, 0.30, 0.50, 1.0]
)

ship_discount_summary = (
    tmp.groupby('ship_ratio_bin')
    .agg(
        orders=('order_id','count'),
        median_margin=('profit_margin','median'),
        mean_margin=('profit_margin','mean')
    )
)

ship_discount_summary
```

```
C:\Users\ceren\AppData\Local\Temp\ipykernel_27140\2428408178.py:11: FutureWarning: The default of observed=False
is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current b
ehavior or observed=True to adopt the future default and silence this warning.
  tmp.groupby('ship_ratio_bin')
```

Out[202…

ship_ratio_bin	orders	median_margin	mean_margin
(0.0, 0.05]	5675	0.25	0.23
(0.05, 0.1]	17111	0.25	0.23
(0.1, 0.15]	8988	0.25	0.23
(0.15, 0.2]	4140	0.25	0.23
(0.2, 0.3]	2965	0.25	0.24
(0.3, 0.5]	1070	0.26	0.23
(0.5, 1.0]	11	0.18	0.17

Shipping Cost Robustness Check

shipping cost is not a primary driver of profitability issues, further confirming that excessive discounting is the main cause of losses.

In [208…

```
df_margin['ship_mode'].value_counts()
```

Out[208…

```
ship_mode
Standard Class    30774
Second Class      10309
First Class        7505
Same Day           2701
Name: count, dtype: int64
```

In [209…

```
(df_margin.groupby('ship_mode')
 .agg(orders=('order_id','count'),
      avg_discount=('discount','mean'),
      avg_margin=('profit_margin','mean'),
      median_margin=('profit_margin','median'),
      profit=('profit','sum'))
 .sort_values('profit'))
```

Out[209…

ship_mode	orders	avg_discount	avg_margin	median_margin	profit
Same Day	2701	0.14	0.05	0.16	76,173.07
First Class	7505	0.15	0.03	0.17	208,104.68
Second Class	10309	0.14	0.05	0.17	294,161.06
Standard Class	30774	0.14	0.05	0.17	890,597.13

In [210…

```
(df_margin.groupby('segment')
 .agg(orders=('order_id','count'),
      sales=('sales','sum'),
      profit=('profit','sum'),
      avg_discount=('discount','mean'),
      median_margin=('profit_margin','median'))
 .sort_values('profit'))
```

Out[210…

segment	orders	sales	profit	avg_discount	median_margin
Home Office	9343	2309956	277,009.18	0.14	0.17
Corporate	15429	3824808	442,785.86	0.14	0.17
Consumer	26517	6508141	749,240.89	0.14	0.17

Operational Checks (Shipping Mode & Segment)

Profit margins and average discounts are very similar across both **shipping modes** and **customer segments**.
This indicates that the profitability issue is **not driven by operational factors or customer type**, further supporting that **discount levels are the main driver of losses**.

Key Insights

1. **24% of all orders are unprofitable**, meaning roughly 1 in 4 orders generates a loss.
2. **Discount levels are strongly negatively correlated with profit margin (-0.85)**, indicating that aggressive discounting significantly reduces profitability.
3. Profit margins typically turn **negative when discounts exceed ~20–25%**, suggesting a clear threshold where promotions become financially harmful.
4. High-discount orders represent **~14% of total sales but generate a disproportionate share of total losses.**
5. Operational factors such as **shipping mode and customer segment show similar margins and discount levels**, indicating they are not key drivers of the profitability problem.

Business Recommendations

Based on the analysis, the following actions are recommended:

1. **Introduce a global discount cap around 20–25%.**
   Discounts above this threshold consistently lead to negative profit margins.
2. **Require approval for high-discount promotions (>30%).**
   These discounts frequently generate losses and should be limited to strategic campaigns.
3. **Review high-discount product categories.**
   Some categories such as Tables and Bookcases experience significant losses when large discounts are applied.
4. **Monitor discount impact using margin-based KPIs.**
   Profit margin should be tracked alongside sales volume to ensure promotions remain profitable.

In [218…

```
#Export Dataset for Dashboard
df_margin.to_csv("cleaned_superstore.csv", index=False)
```